

Interrupciones de 8086

Ahora que hemos aceptado que las interrupciones son parte del funcionamiento normal de un procesador, veamos el mecanismo de procesamiento de interrupciones del 8086. 8086 tiene interrupciones de hardware, interrupciones de software e interrupciones generadas por errores. Para los tres casos mencionados aquí, el mecanismo de interrupción es diferente, pero la forma en que responde el procesador es similar. Entonces, primero echemos un vistazo a la forma en que 8086 responde a cualquier interrupción. El punto a tener en cuenta es que, después de procesar una solicitud de interrupción, el procesador tiene que volver a su tarea anterior que quedó sin terminar. Por lo tanto, una interrupción no es muy diferente de una instrucción CALL en su filosofía, pero existen algunas diferencias en la forma en que se maneja y procesa.

Respuesta de interrupción de 8086

Después de cada ciclo de instrucción, el procesador verifica si hay alguna interrupción esperando servicio. Si encuentra una solicitud de interrupción y decide reconocerla y atenderla, la respuesta es la siguiente secuencia de pasos:

- i) El registro de bandera se coloca en la pila.
- ii) El indicador de interrupción está deshabilitado ($IF = 0$).
- iii) El indicador de trampa está deshabilitado ($TF = 0$).
- iv) El registro CS se coloca en la pila.
- v) El registro IP se coloca en la pila.
- vi) Se transfiere el control al lugar en el que se realice la correspondiente "Interrupción del Servicio". Se almacena la rutina (ISR). En efecto, esto sería un gran salto.
- vii) Se ejecuta el programa correspondiente al ISR. La última instrucción en el ISR será IRET.
- viii) Luego, la IP se extrae de la pila.

- ix) CS sale de la pila.
- x) El registro Bandera se extrae de la pila.
- xi) El control vuelve al punto en el que lo había dejado.

Los pasos del i al vi constituyen las acciones antes de llegar a la rutina del servicio de interrupción, y los pasos del viii al xi ocurren al regresar de la interrupción.

Entremos en los detalles de los pasos de la lista anterior:

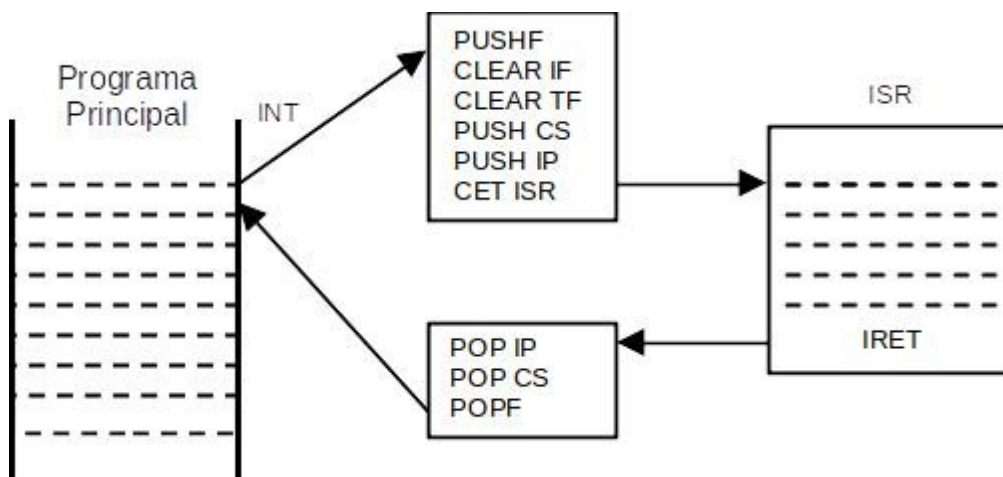


Figura 1: Pasos en el procesamiento de una interrupción

El primer paso es parte de la acción de guardar el "contexto" del programa actual. Primero, el registro de estado (banderas) se guarda en la pila para su posterior recuperación. Esto se debe a que el ISR puede cambiar el estado de las banderas.

El segundo paso es garantizar que esta rutina de interrupción no sea interrumpida por otras interrupciones recibidas en la línea INTR.

El tercer paso es garantizar que el programa iniciado no se detenga después de la ejecución de cada instrucción (esto sucede si el indicador Trap está activado).

El cuarto paso y el quinto paso son para guardar en la pila, la dirección de la siguiente instrucción en la secuencia actual, tal como se hace en el caso de una CALL lejana. Serán retirados de la pila una vez finalizado el procesamiento de la interrupción.

Tanto IP como CS se guardan porque el nuevo programa adoptado estará en un segmento de código diferente.

El sexto paso es el de transferir el control a una ubicación en un segmento de código diferente. Por eso se le denomina "salto de larga distancia".

La rutina del servicio de interrupción, que es el programa en el que se produjo el salto, ahora se retomará para su ejecución. La última instrucción de un ISR es una instrucción IRET que provocará la reversión de las acciones anteriores.

En los pasos octavo y noveno, primero se extrae de la pila la IP y luego la CS. Así la dirección de la siguiente instrucción en la secuencia normal vuelve a estar en los registros requeridos.

En el décimo paso, el registro de banderas se elimina y, por lo tanto, el estado original de todas las banderas es restaurado.

Finalmente, la ejecución de la instrucción se reanuda desde donde había sido desviada.

Rutina de servicio de interrupción y vector de interrupción

Ahora bien, ¿qué es una rutina de servicio de interrupción? Cuando ocurre una interrupción, el procesador suspende la ejecución de su tarea actual y asume otra tarea según lo requiera la fuente de interrupción. Este programa, o rutina como se le puede llamar, se denomina "rutina de servicio de interrupción". Esta rutina corresponde a la solicitud de una fuente particular de interrupción y también se denomina "manejador de interrupciones". Esto significa que para cualquier interrupción que se produzca, existe una rutina de servicio de interrupción (ISR) particular.

Ahora bien, ¿dónde está disponible este ISR? Debe estar disponible en la memoria y se debe acceder a él cuando ocurre la interrupción específica. Para ello se debe obtener la dirección del ISR. La dirección de un ISR se denomina "vector de interrupción". Para un sistema basado en 8086, cualquier dirección de código tiene el siguiente formato, CS:IP. Por lo tanto, el vector de interrupción para cualquier interrupción tiene 4 bytes: dos para el

valor CS y dos para el valor IP. Por lo tanto, si se obtiene el vector de interrupción para una interrupción particular, el control se puede transferir a la nueva ubicación utilizando los nuevos valores de CS e IP especificados como el "vector de interrupción".

Es obvio que el número de vectores de interrupción en un sistema es el mismo que el número de interrupciones que el sistema puede procesar. El 8086 tiene 256 vectores de interrupción y desde cada vector se especifica en 4 bytes, implica que se asignan $256 \times 4 = 1024$ bytes (1K) de memoria para almacenar los vectores de interrupción. Estos 256 vectores se almacenan en una tabla llamada "Tabla de vectores de interrupción" (IVT) en la RAM del sistema desde las ubicaciones 00000 a 003FFH, es decir, hasta 0000: 03FF. Cada vector de interrupción está numerado del 0 al 255 y, como tal, cada interrupción también está numerada del mismo modo.

Estos números resultan ser, por tanto, lo que se puede denominar "números de tipo" de las interrupciones. Por lo tanto, hay interrupciones designadas como Int 0, Int 1, Int 2... Int 255. Estos números muestran la posición del vector de interrupción correspondiente en la tabla de vectores de interrupción. Así, el vector de INT 0 es la 1ª entrada en el IVT el de INT 1 es la segunda entrada... el de INT 6 es la 7ª entrada y así sucesivamente. Dado que cada vector de interrupción tiene 4 bytes de longitud, la ubicación de memoria correspondiente al vector de INT n se obtiene como $n \times 4$, es decir, para INT 1, la dirección del primer byte de su vector es $1 \times 4 = 0004$, para INT 0 es 0000 y así sucesivamente. De hecho, así es como reacciona el 8086 ante una interrupción con un número de tipo. Multiplica el número de tipo por 4, obtiene la dirección de su vector de interrupción, carga IP y CS con estos nuevos valores y comienza a ejecutar el ISR que ha localizado.

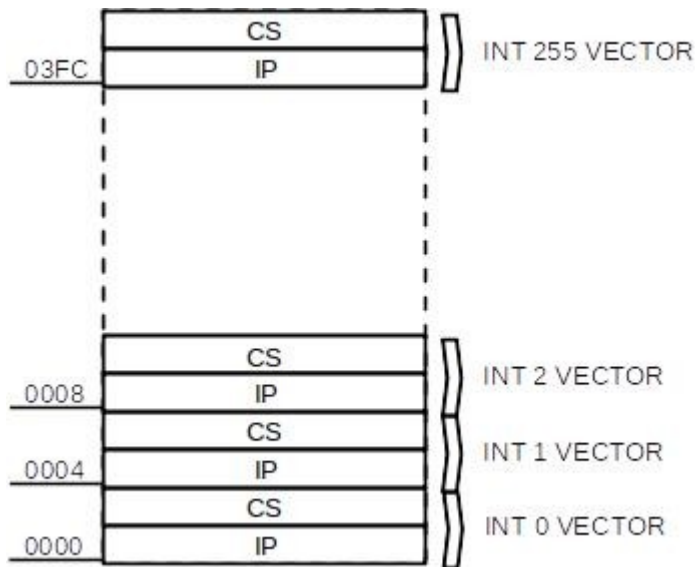


Figura 2: Tabla Vector de Interrupciones del 8086

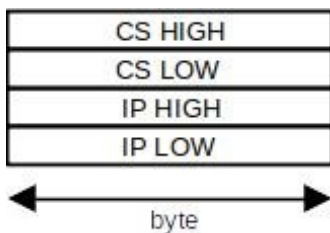


Figura 3: Una entrada típica en el vector de interrupción (4 bytes)

Ejemplo

Encuentre la dirección (en el IVT) del vector de interrupción de INT 61H. Encuentre la dirección física del ISR correspondiente a esta interrupción si el vector es 0F00:9872.

Solución

El número de tipo de interrupción es 61H = 97 en decimal.

La dirección del vector de interrupción es $97 \times 4 = 388 = 184H$.

Por lo tanto, el vector de interrupción debe almacenarse en el IVT en la ubicación 0000:0184 en adelante.

Para ISR, el valor de CS es 0F00H y el valor de IP es 9872H. Se muestra la parte de la tabla de vectores de interrupción que tiene estos vectores almacenados.

	0FH
0186H	00
	98H
0184H	72H

Dirección
(address)

Figura 4: Vector de interrupción para INT61H

Ahora bien, ¿cómo obtiene estas entradas la tabla de vectores de interrupción?

Se mencionó que después del "encendido", el procesador se activa en la dirección FFFF0H. Este es el punto de entrada al BIOS en la ROM. BIOS contiene un conjunto de rutinas que brindan soporte para dispositivos. Estas rutinas verifican, verifican e inician varios dispositivos y también establecen dos áreas de datos, una de las cuales es la tabla de vectores de interrupción desde las ubicaciones 00000 a 003FFH.

Nota

Texto extraído y traducido del libro The X86 Microprocessors de Lyla B. Das, página 250.